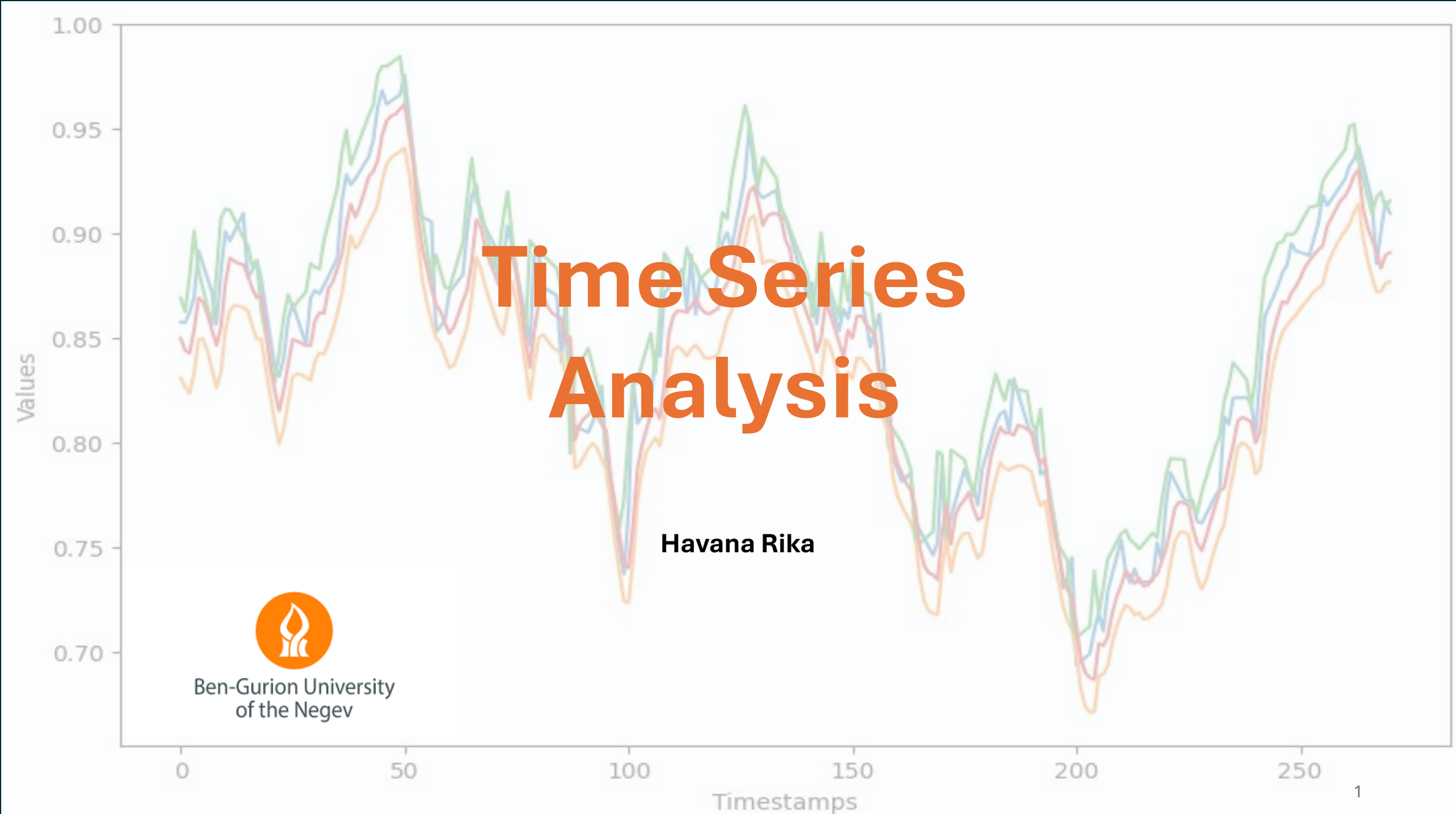


Time Series Analysis

Havana Rika



Ben-Gurion University
of the Negev



Machine Learning Models for Time-Series

- Additional machine learning approaches for forecasting
- Focus on practical and competitive methods
- **kNN** with Dynamic Time Warping (DTW)
- **Silverkite** forecasting framework
- Gradient Boosting with **XGBoost**
- **Ensemble** forecasting with Kats

Motivation

- Classical models are not always sufficient
- Machine learning captures **complex patterns**
- Many specialized **forecasting libraries** exist
- Modern tools provide **automation** and **scalability**
- **Feature engineering** remains critical

Popular Time-Series Libraries

- TSMFRESH: automated feature extraction
- SKTime: time-series machine learning framework
- Greykite: forecasting library from LinkedIn
- Kats: forecasting framework from Meta

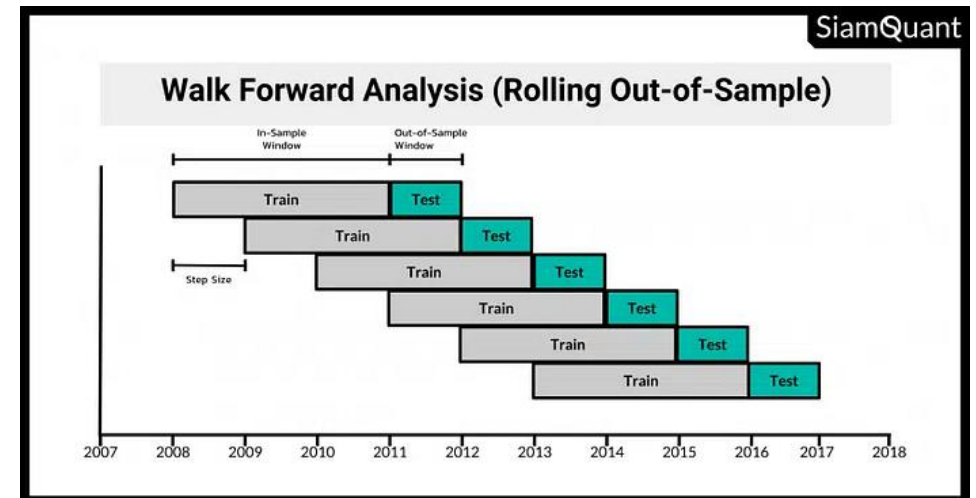
Libraries support forecasting and model selection

Validation in Time-Series

- **Standard** k-fold validation may be **misleading**
- Time-series data **evolves** over time
- **Future** data must remain **unseen** during training
- **Temporal ordering** must be preserved
- **Prevents** overly **optimistic** evaluation

Walk-Forward Validation

- Train on historical observations
- Test on future observations
- Move the training window forward
- Repeat across multiple folds
- Produces realistic performance estimates
- Reduces risk of overfitting



Walk-Forward Validation

In terms of training, validation, and test datasets, this means that we'll **adjust model parameters** entirely on **training** and **validation** datasets, and we'll **benchmark** our test based on a set of data that's **more advanced** in time

1) Cross-Validation (model tuning)

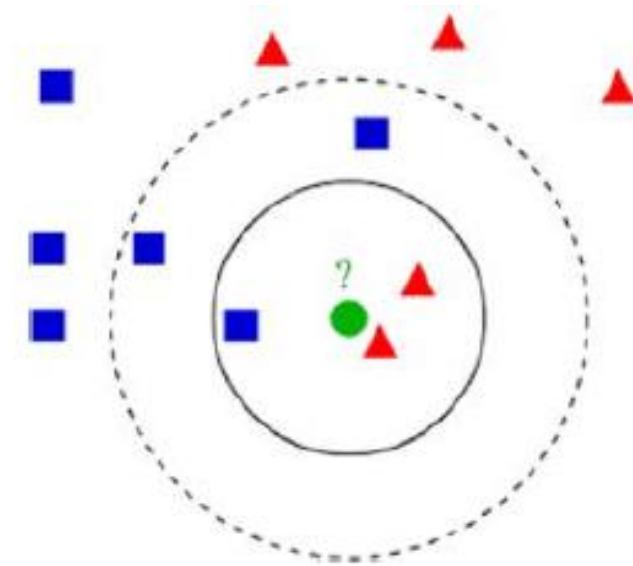


2) Final Evaluation



K-Nearest Neighbors (KNN)

- Instance-based learning algorithm
- Finds similar historical examples
- Makes predictions from nearest neighbors
- Simple but surprisingly competitive



Need to set:

- The **number of neighbors** (k) you want to base your output on
- The **integration** function (for example, the average or the most frequent value)
- The **distance function** to use to find the nearest data points

Dynamic Time Warping (DTW) & KNN

- Measures **similarity** between **time-series**
 - **Handles** temporal **misalignment**
 - Supports sequences of **different lengths**
 - Often **outperforms Euclidean** distance
-
- **DTW** used as distance function in **KNN**
 - Finds **similar temporal patterns**
 - Effective for **classification** tasks
 - Historically **strong** benchmark **method**
 - **Computationally expensive**

The Robot Execution Failures Dataset

index	id	time	F_x	F_y	F_z	T_x	T_y	T_z
0	1	0	-1	-1	63	-3	-1	0
1	1	1	0	0	62	-3	-1	0
2	1	2	-1	-1	61	-3	0	0
3	1	3	-1	-1	63	-2	-1	0
4	1	4	-1	-1	63	-3	-1	0

```
df_ts.shape  
(1320, 8)
```

each robot execution has an id, a time column, and six sensor
signals: F_x, F_y, F_z, T_x, T_y, T_z

Meaning of the columns

Column	Meaning
id	The robot execution sample ID
time	The time step inside that sample
F_x	Force measurement on x-axis
F_y	Force measurement on y-axis
F_z	Force measurement on z-axis
T_x	Torque measurement on x-axis
T_y	Torque measurement on y-axis
T_z	Torque measurement on z-axis

The Target Labels

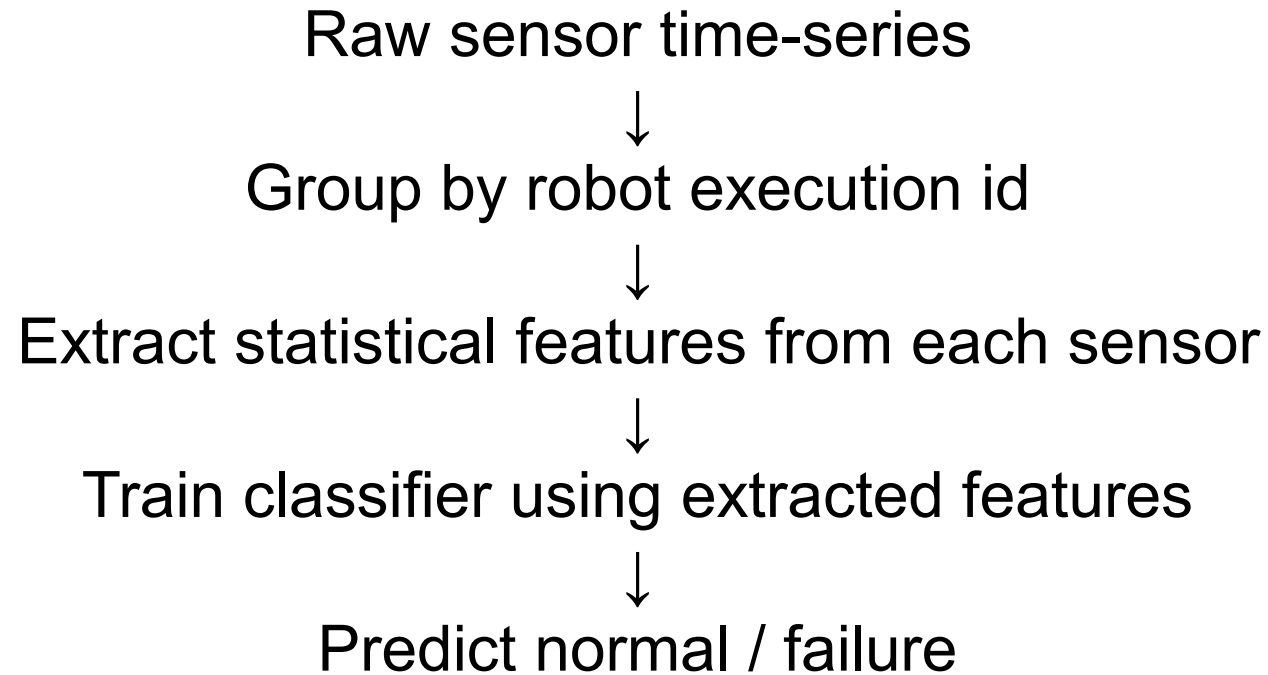
id	y
1	False
2	False
3	False
4	False
5	False

It contains one label per robot execution id

True = normal execution

False = failure / non-normal execution

The Pipeline



KNN with DTW in Python

```
pip install tsfresh tslearn
```

```
from tsfresh.examples import load_robot_execution_failures
from tsfresh.examples.robot_execution_failures import download_robot_
execution_failures
```

```
download_robot_execution_failures()
df_ts, y = load_robot_execution_failures()
```

```
print(f"{y.mean():.2f}")
```

The mean of y is 0.24.

```
from tsfresh import extract_features
from tsfresh import select_features
from tsfresh.utilities.dataframe_functions import impute

extracted_features = impute(extract_features(df_ts, column_id="id",
column_sort="time"))
features_filtered = select_features(extracted_features, y)
```

```
from sklearn.model_selection import TimeSeriesSplit, GridSearchCV
from tslearn.neighbors import KNeighborsTimeSeriesClassifier
```

```
knn = KNeighborsTimeSeriesClassifier()
param_search = {
    'metric' : ['dtw'],
    'n_neighbors': [1, 2, 3]
}
tscv = TimeSeriesSplit(n_splits=2)
```

```
gsearch = GridSearchCV(
    estimator=knn,
    cv=tscv,
    param_grid=param_search
)
gsearch.fit(
    features_filtered,
    y
)
```

Forecast Evaluation

- Compare forecast vs actual values
- RMSE
- MSE
- MAE
- R^2
- Correlation metrics

Linear regression

$$\hat{y} = \beta_0 + \sum_{i=1}^k X_i \beta_i$$

$$RSS = \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Commonly, five assumptions are associated with linear regression, as follows:

- The **relationship** between the **independent** and **dependent** variables is **linear**.
- The **errors** are **normally distributed**.
- The **variance** of the **errors** is **constant** across all the values of the independent variable.
- There is **no autocorrelation** in the **errors**.
- There is little to **no correlation** between **independent variables** (multi-collinearity).

Linear regression

```
from sklearn.linear_model import LinearRegression
model_config = ModelConfig(
    model=LinearRegression(),
    name="Linear Regression",
    # LinearRegression is sensitive to normalized data
    normalize=True,
    # LinearRegression cannot handle missing values
    fill_missing=True,
)
y_pred, metrics, feat_df = evaluate_model(
    model_config,
    feat_config,
    missing_value_config,
    train_features,
    train_target,
    test_features,
    test_target,
)
```



Helper functions for evaluating models

```
def evaluate_model(
    model_config,
    feature_config,
    missing_config,
    train_features,
    train_target,
    test_features,
    test_target,
):
    ml_model = MLForecast(
        model_config=model_config,
        feature_config=feature_config,
        missing_config=missing_value_config,
    )
    ml_model.fit(train_features, train_target)
    y_pred = ml_model.predict(test_features)
    feat_df = ml_model.feature_importance()
    metrics = calculate_metrics(test_target, y_pred, model_config.name, train_target)
    return y_pred, metrics, feat_df
```

FeatureConfig

FeatureConfig is a Python dataclass that defines a few key attributes and functions that are necessary while processing the data. For instance, continuous, categorical, and Boolean columns need separate kinds of preprocessing before being fed into the machine learning model.

FeatureConfig holds:

- date
- target
- original_target
- continuous_features
- categorical_features
- boolean_features
- index_cols
- exogenous_features

FeatureConfig

In addition to a bit of validation on the inputs, there is also a helpful method called `get_X_y` in the class, with the following parameters:

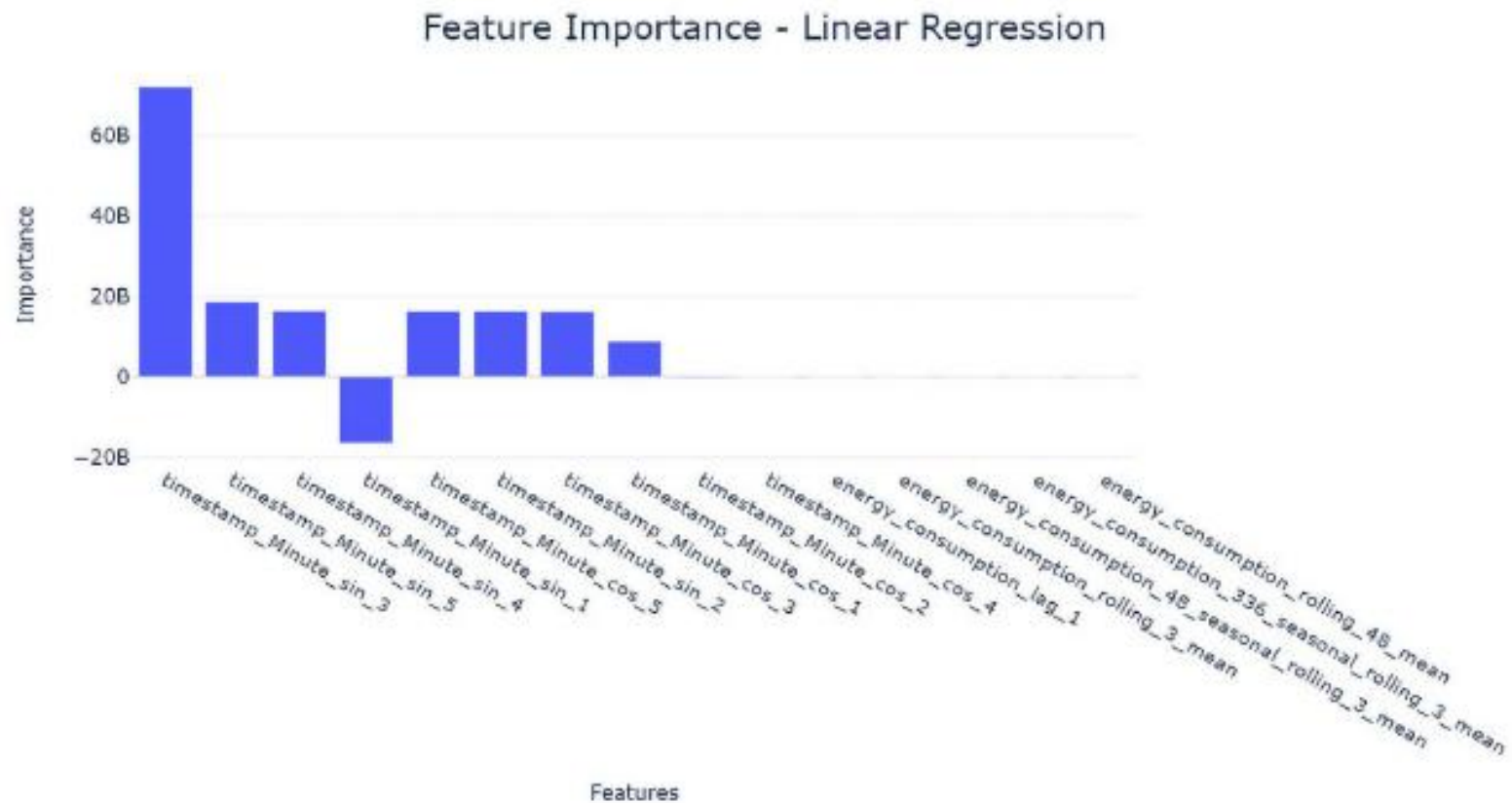
- **df**: A DataFrame that contains all the necessary columns, including the target, if available
- **categorical**: A Boolean flag for including categorical features or not
- **exogenous**: A Boolean flag for including exogenous features or not

The function returns a tuple of (**features**, **target**, **original_target**).

After setting the `FeatureConfig` data class, we can pass any DataFrame with the features defined to the `get_X_y` function to get the features, target, and original target:

```
train_features, train_target, train_original_target = feat_config.get_X_y(  
    sample_train_df, categorical=False, exogenous=False  
)
```

coef_ attribute of a trained scikitlearn model



Regularized linear regression

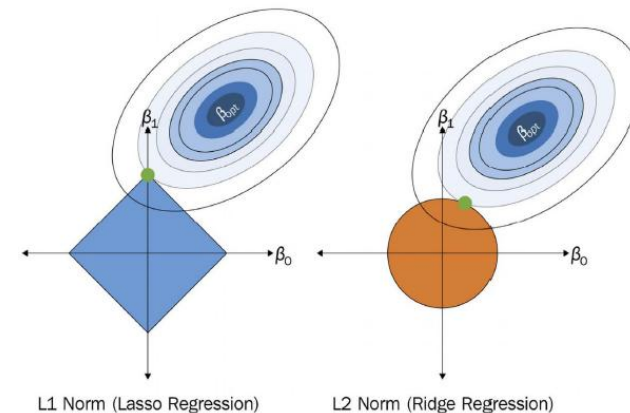
$$RSS = \sum_{i=1}^N (y_i - \hat{y}_i)^2 + \lambda W$$

The L1 norm is defined as the sum of the absolute values of the matrix. For weight decay regularization, the L1 norm would be as follows:

$$W = \sum_{i=1}^k |\beta_i|$$

The L2 norm is defined as the sum of squared values of a matrix. For weight decay regularization, the L2 norm would be as follows:

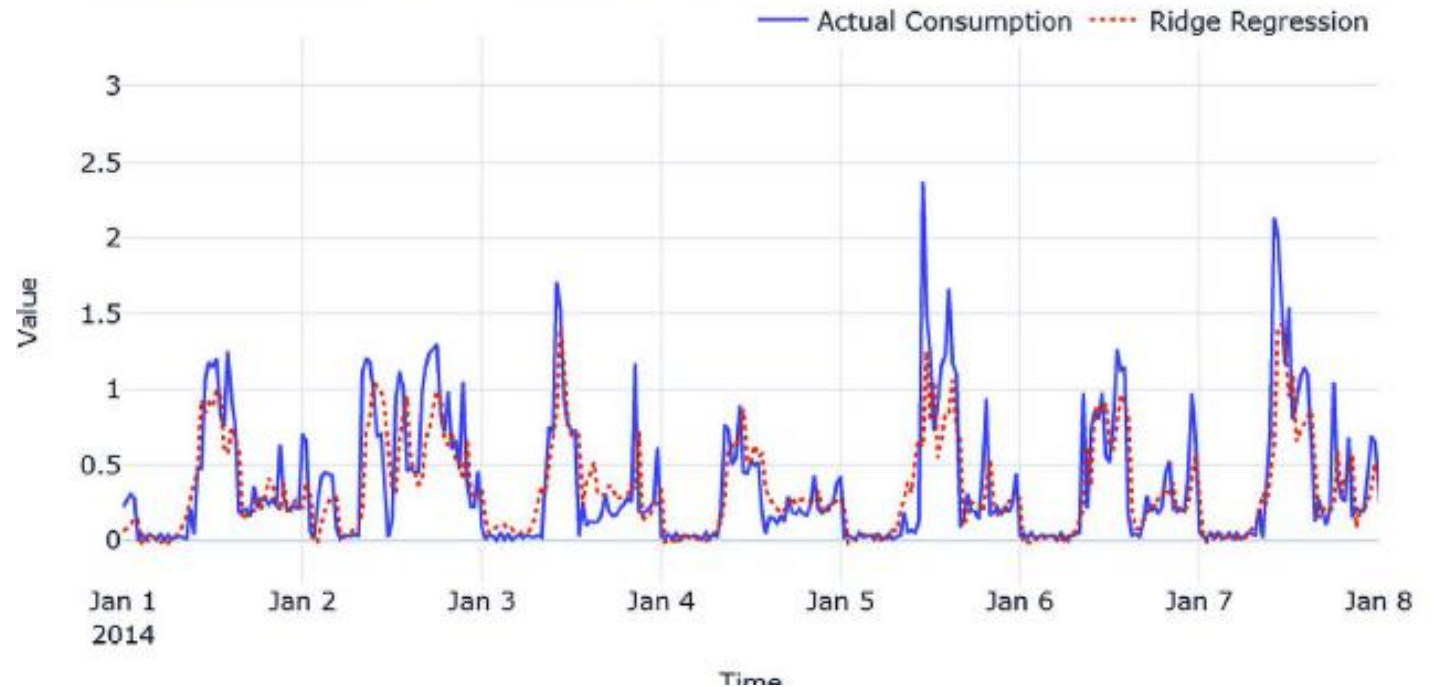
$$W = \sum_{i=1}^k |\beta_i^2|$$

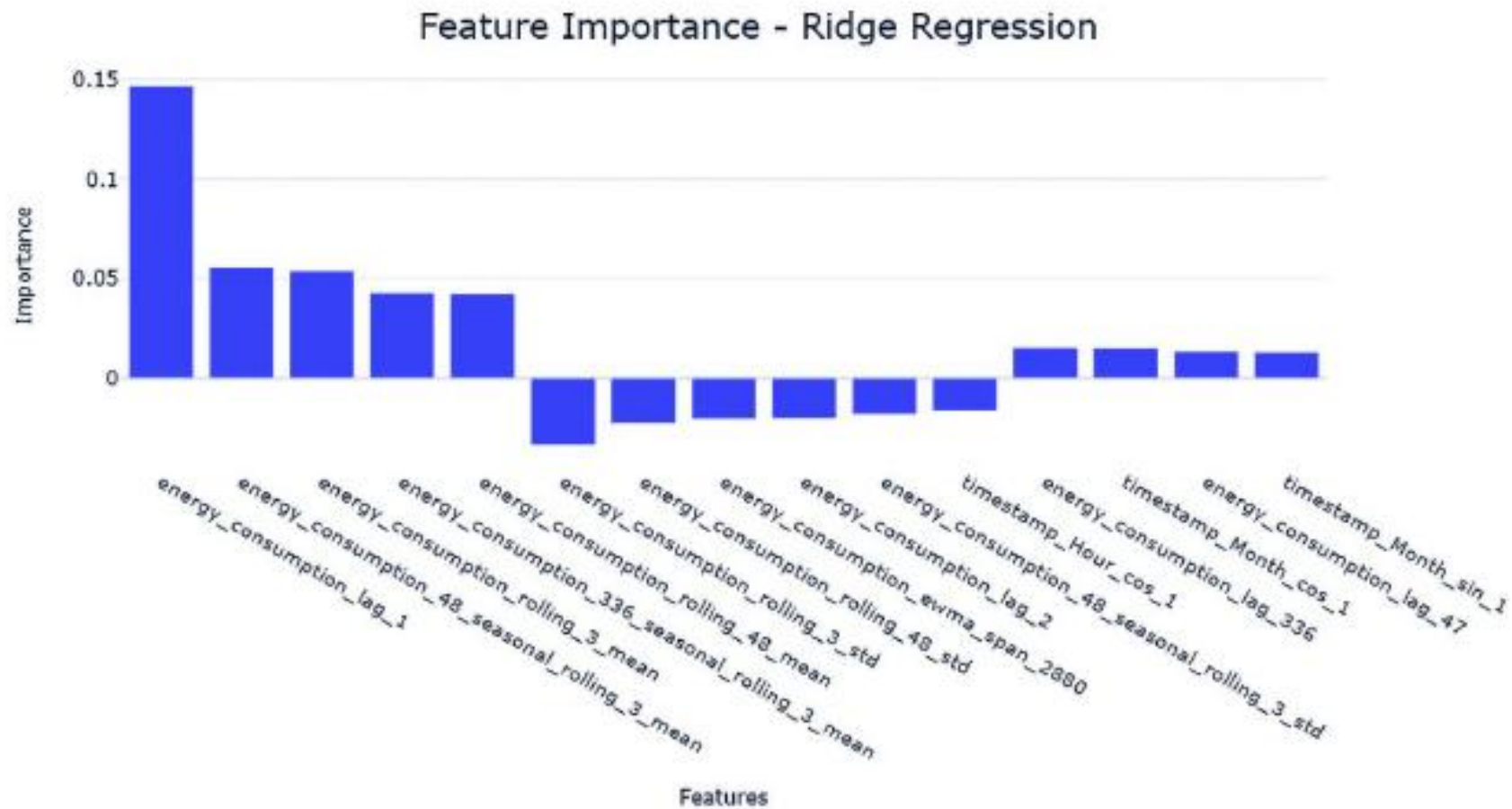


Ridge Regression

```
from sklearn.linear_model import RidgeCV
model_config = ModelConfig(
    model=RidgeCV(),
    name="Ridge Regression",
    # RidgeCV is sensitive to normalized data
    normalize=True,
    # RidgeCV does not handle missing values
    fill_missing=True
)
y_pred, metrics, feat_df = evaluate_model(
    model_config,
    feat_config,
    missing_value_config,
    train_features,
    train_target,
    test_features,
    test_target,
)
```

Ridge Regression: MAE: 0.1595 | MSE: 0.0748 | MASE: 1.2430 | Bias: 6.1637

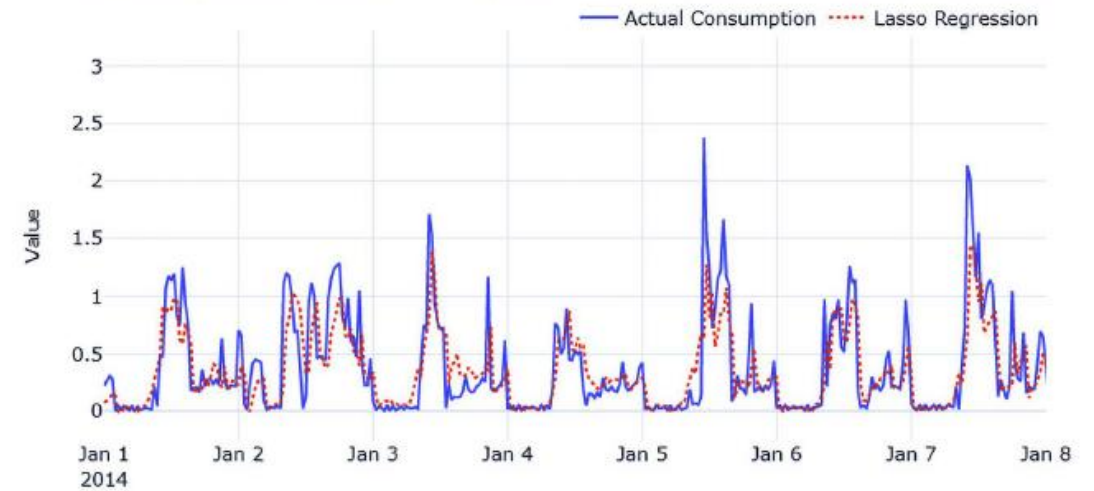




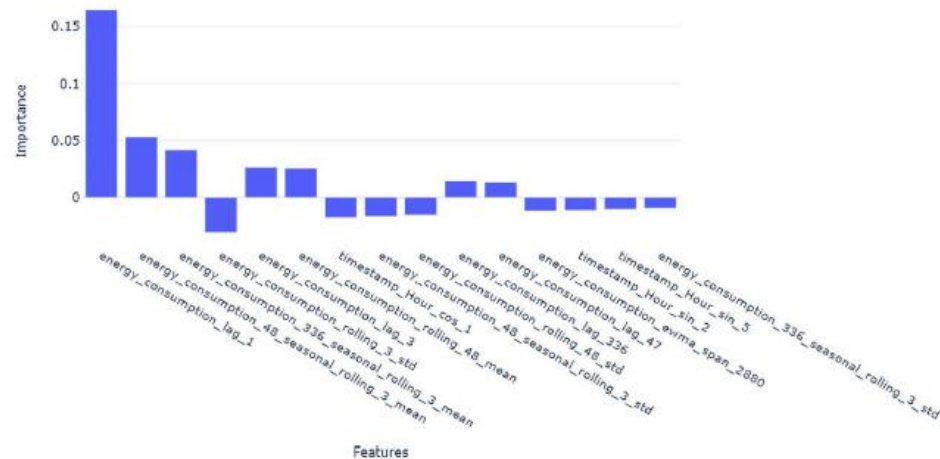
lasso regression

```
from sklearn.linear_model import LassoCV
model_config = ModelConfig(
    model=LassoCV(),
    name="Lasso Regression",
    # LassoCV is sensitive to normalized data
    normalize=True,
    # LassoCV does not handle missing values
    fill_missing=True
)
y_pred, metrics, feat_df = evaluate_model(
    model_config,
    feat_config,
    missing_value_config,
    train_features,
    train_target,
    test_features,
    test_target,
)
```

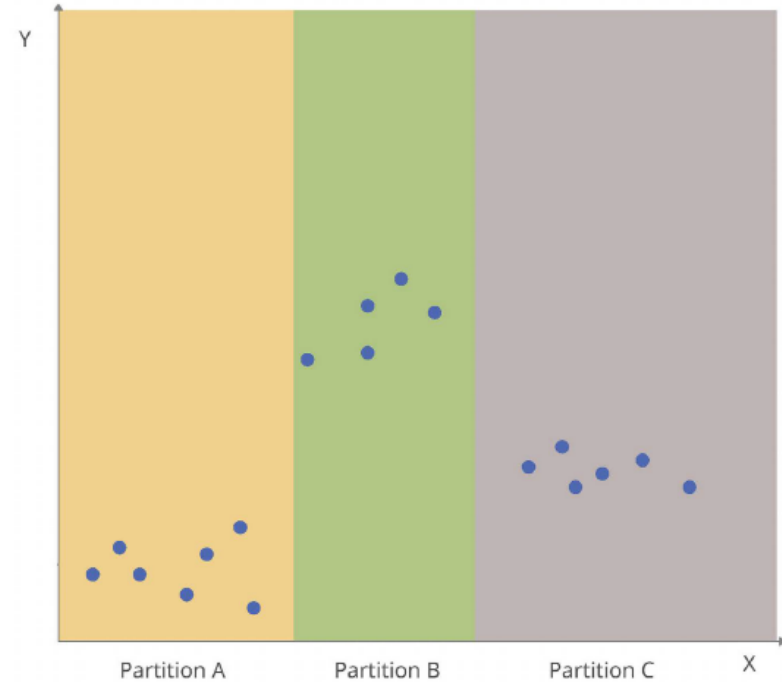
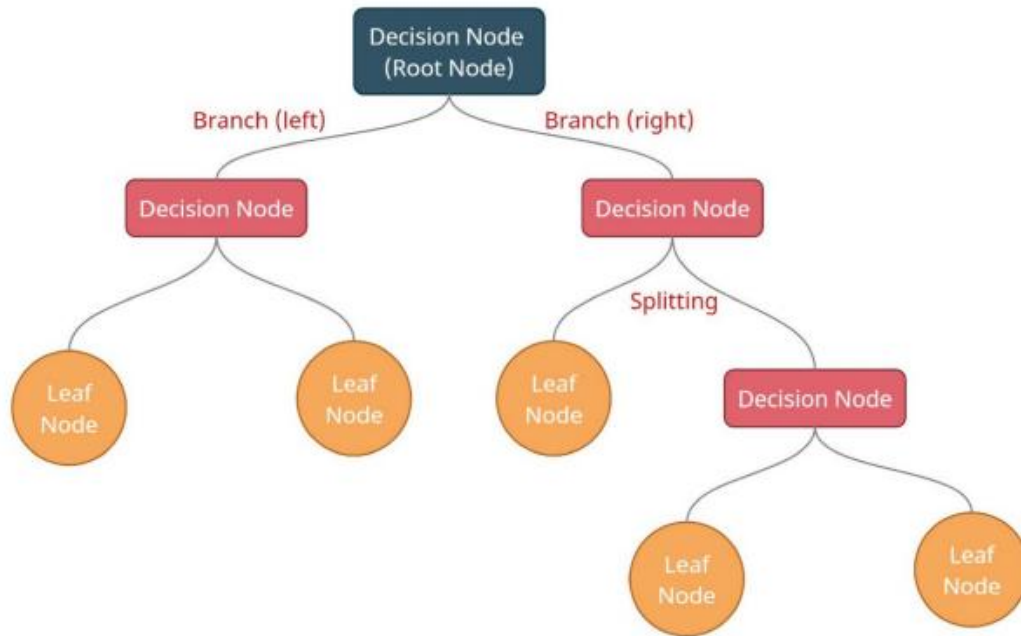
Lasso Regression: MAE: 0.1599 | MSE: 0.0743 | MASE: 1.2463 | Bias: 3.6669



Feature Importance - Lasso Regression



Decision trees

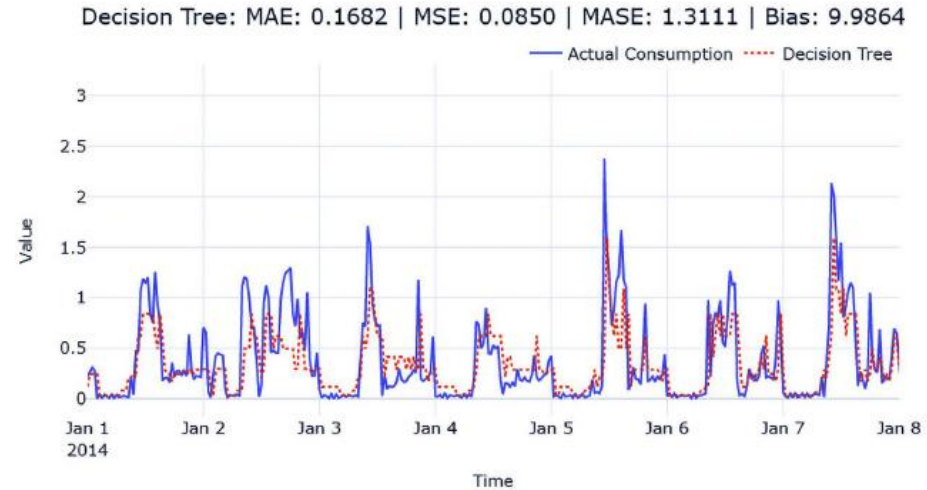


Formally, we can define the function that's been generated by a decision tree that has M partitions, $P_1, P_2, \dots, P_M$, as follows:

$$\hat{y} = \sum_{m=1}^M c_m I(x \in P_m)$$

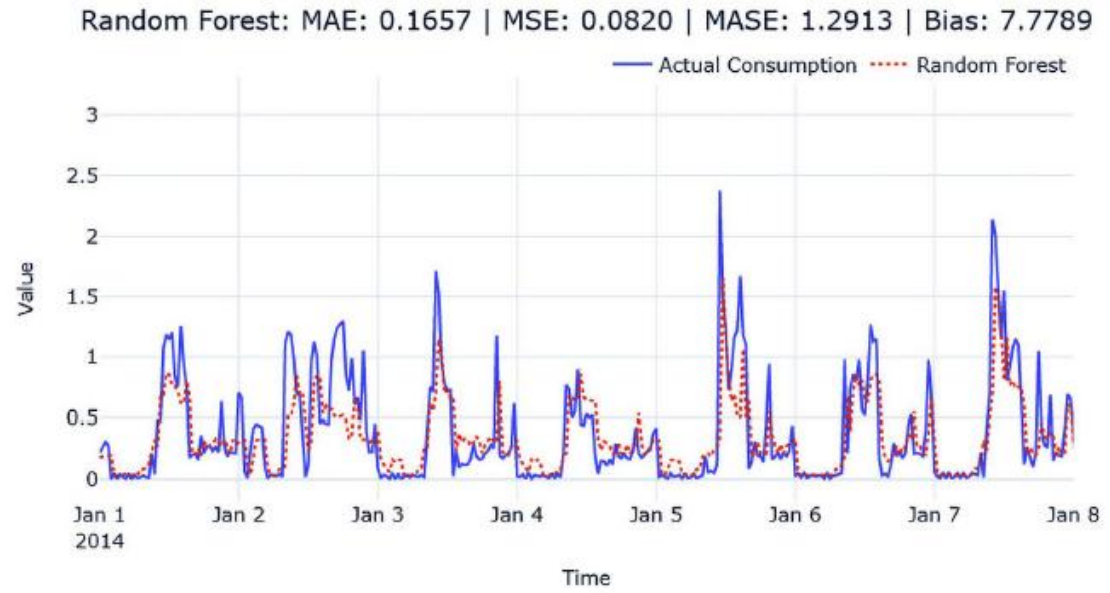
Decision trees

```
from sklearn.tree import DecisionTreeRegressor
model_config = ModelConfig(
    model=DecisionTreeRegressor(max_depth=4, random_state=42),
    name="Decision Tree",
    # Decision Tree is not affected by normalization
    normalize=False,
    # Decision Tree in scikit-learn does not handle missing values
    fill_missing=True,
)
y_pred, metrics, feat_df = evaluate_model(
    model_config,
    feat_config,
    missing_value_config,
    train_features,
    train_target,
    test_features,
    test_target,
)
```



Random forest

```
from sklearn.ensemble import RandomForestRegressor
model_config = ModelConfig(
    model=RandomForestRegressor(random_state=42, max_depth=4),
    name="Random Forest",
    # RandomForest is not affected by normalization
    normalize=False,
    # RandomForest in scikit-learn does not handle missing values
    fill_missing=True,
)
y_pred, metrics, feat_df = evaluate_model(
    model_config,
    feat_config,
    missing_value_config,
    train_features,
    train_target,
    test_features,
    test_target,
)
```



Gradient Boosting (XGBoost)

- Ensemble learning method
- Sequentially improves previous models
- Each tree reduces residual errors
- Strong predictive performance
- Widely used in forecasting applications

XGBoost can be used for forecasting, by training the model based on past values to predict future values, and this can be applied to univariate as well as multivariate time-series.

XGBoost for Forecasting

```
pip install xgboost
```

```
import pandas as pd

owid_covid = pd.read_csv("https://covid.ourworldindata.org/data/owid-covid-data.csv")
owid_covid["date"] = pd.to_datetime(owid_covid["date"])
df = owid_covid[owid_covid.location == "France"].set_index("date",
drop=True).resample('D').interpolate(method='linear').reset_index()
```

```
print(owid_covid.shape)
```

```
(429435, 67)
```

owid-covid-data

`owid-covid-data.csv` is a **global COVID-19 panel dataset** from **Our World in Data**. It is arranged as:

one row = one location on one date

iso_code	continent	location	date	total_cases	new_cases	total_deaths	new_deaths	population
FRA	Europe	France	2020-03-15	5423	838	127	36	67813000
FRA	Europe	France	2020-03-16	6633	1210	148	21	67813000

Main identifier columns

Column	Meaning
<code>iso_code</code>	Country/region code, for example <code>FRA</code> , <code>USA</code> , <code>OWID_WRL</code>
<code>continent</code>	Continent, for example <code>Europe</code>
<code>location</code>	Country or region name
<code>date</code>	Observation date

Important COVID case columns

Column	Meaning
<code>total_cases</code>	Total confirmed cases up to that date
<code>new_cases</code>	New confirmed cases on that date
<code>new_cases_smoothed</code>	7-day smoothed new cases
<code>total_cases_per_million</code>	Total cases per 1,000,000 people
<code>new_cases_per_million</code>	New cases per 1,000,000 people
<code>new_cases_smoothed_per_million</code>	Smoothed new cases per 1,000,000 people

Important death columns

Column	Meaning
<code>total_deaths</code>	Total deaths up to that date
<code>new_deaths</code>	New deaths on that date
<code>new_deaths_smoothed</code>	7-day smoothed new deaths
<code>total_deaths_per_million</code>	Total deaths per 1,000,000 people
<code>new_deaths_per_million</code>	New deaths per 1,000,000 people
<code>new_deaths_smoothed_per_million</code>	Smoothed new deaths per 1,000,000 people

Hospital and ICU columns

Column	Meaning
<code>icu_patients</code>	Current COVID patients in ICU
<code>hosp_patients</code>	Current COVID patients in hospital
<code>weekly_icu_admissions</code>	Weekly new ICU admissions
<code>weekly_hosp_admissions</code>	Weekly new hospital admissions
<code>_per_million</code> versions	Same metrics normalized by population

Testing columns

Column	Meaning
<code>total_tests</code>	Total COVID tests
<code>new_tests</code>	New tests
<code>new_tests_smoothed</code>	7-day smoothed new tests
<code>positive_rate</code>	Share of tests that are positive
<code>tests_per_case</code>	Number of tests per confirmed case
<code>tests_units</code>	Whether tests are people tested, tests performed, or samples tested

Vaccination columns

Column	Meaning
<code>total_vaccinations</code>	Total vaccine doses administered
<code>people_vaccinated</code>	People with at least one dose
<code>people_fully_vaccinated</code>	People fully vaccinated
<code>total_boosters</code>	Booster doses
<code>new_vaccinations</code>	New doses
<code>new_vaccinations_smoothed</code>	7-day smoothed new doses
<code>_per_hundred</code> versions	Same metrics per 100 people

Demographic and background columns

Column	Meaning
population	Population
population_density	People per square kilometer
median_age	Median age
aged_65_older	Share aged 65+
aged_70_older	Share aged 70+
gdp_per_capita	GDP per capita
diabetes_prevalence	Diabetes prevalence
life_expectancy	Life expectancy
human_development_index	HDI

XGBoost Forecasting Pipeline

- Feature extraction
- Data transformation
- XGBoost regression model
- Training on historical observations
- Prediction of future values
- Performance evaluation

Date Feature Transformer

- Automated extraction of calendar features
- Implemented as a Scikit-Learn transformer
- Integrates into machine learning pipelines
- Simplifies preprocessing workflow
- Enables reusable forecasting pipelines

XGBoost for Forecasting

```
from sklearn.base import TransformerMixin, BaseEstimator

class DateFeatures(TransformerMixin, BaseEstimator):
    features = [
        "hour",
        "year",
        "day",
        "weekday",
        "month",
        "quarter",
    ]

    def __init__(self):
        super().__init__()

    def transform(self, df: pd.DataFrame):
        Xt = []
        for col in df.columns:
            for feature in self.features:
                date_feature = getattr(
                    getattr(
                        df[col], "dt"
                    ), feature
                )
```

```
                )
                date_feature.name = f"{col}_{feature}"
                Xt.append(date_feature)

        df2 = pd.concat(Xt, axis=1)
        return df2

    def fit(self, df: pd.DataFrame, y=None, **fit_params):
        return self
```

XGBoost for Forecasting

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline, make_pipeline
preprocessor = ColumnTransformer(
    transformers=[(
        "date",
        make_pipeline(
            DateFeatures(),
            ColumnTransformer(transformers=[
                ("cyclical", CyclicalFeatures(),
                ["date_day", "date_weekday", "date_month"]
            )
        ], remainder="passthrough")
    ), ["date"],
    ),], remainder="passthrough"
)
```

```
from xgboost import XGBRegressor
pipeline = Pipeline(
    [
        ("preprocessing", preprocessor),
        ("xgb", XGBRegressor(objective="reg:squarederror", n_
estimators=1000))
    ]
)
```

XGBoost for Forecasting

```
TRAIN_SIZE = int(len(df) * 0.9)
HORIZON = 1
TARGET_COL = "new_cases"
```

```
X_train, X_test = df.iloc[HORIZON:TRAIN_SIZE], df.iloc[TRAIN_
SIZE+HORIZON:]
y_train = df.shift(periods=HORIZON).iloc[HORIZON:TRAIN_SIZE][TARGET_
COL]
y_test = df.shift(periods=HORIZON).iloc[TRAIN_SIZE+HORIZON:][TARGET_
COL]
```

```
FEATURE_COLS = ["date"]
pipeline.fit(X_train[FEATURE_COLS], y_train)
```

```
Pipeline(steps=[('preprocessing',
                  ColumnTransformer(remainder='passthrough',
                                     transformers=[('date',
                                                     Pipeline(steps=[('datefeatures',
                                                                       DateFeatures()),
                                                                       ('columntransformer',
                                                                       ColumnTransformer(remainder='passthrough',
                                                                                   transformers=[('cyclical',
                                                                                   CyclicalFeatures(),
                                                                                   ['date_day',
                                                                                   'date_weekday',
                                                                                   'date_month'])])),
                                                     ['date'])])),
                  ('xgb',
                   XGBRegressor(n_estimators=1000,
                                objective='reg:squarederror'))])
```

XGBoost for Forecasting

```
MAX_HORIZON = 90
X_test_horizon = pd.Series(pd.date_range(
    start=df.date.min(),
    periods=len(df) + MAX_HORIZON,
    name="date"
)).reset_index()
```

```
forecasted = pd.concat(
    [pd.Series(pipeline.predict(X_test_horizon[FEATURE_COLS])),
     pd.Series(X_test_horizon.date)],
    axis=1
)
forecasted.columns = [TARGET_COL, "date"]
```

```
actual = pd.concat(
    [pd.Series(df[TARGET_COL]), pd.Series(df.date)],
    axis=1
)
actual.columns = [TARGET_COL, "date"]
```

```
fig, ax = plt.subplots(figsize=(12, 6))
forecasted.set_index("date").plot(linestyle='--', ax=ax)
```

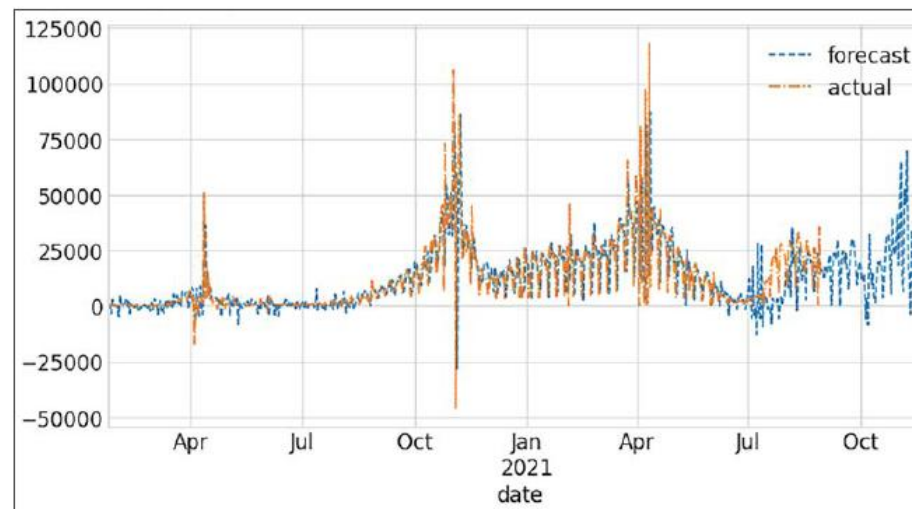


Figure 7.9: Forecast versus actual (XGBoost)

```
from sklearn.metrics import mean_squared_error

test_data = actual.merge(forecasted, on="date", suffixes=("_actual",
    "_predicted"))
mse = mean_squared_error(test_data.new_cases_actual, test_data.new_
    cases_predicted, squared=False) # RMSE
print("The root mean squared error (RMSE) on test set: {:.2f}".
    format(mse))
```

The root mean squared error (RMSE) on test set: 12753.41

Silverkite Overview

- Forecasting framework from LinkedIn
- Designed for speed and accuracy
- Supports multiple seasonalities
- Handles holidays and events
- Suitable for business forecasting



part of the broader [Greykite](#) library and specializes in modeling complex seasonality, event/holiday effects, and autoregressive features at scale

How Silverkite Works?

How Silverkite Works

Silverkite operates by taking a time series and **decomposing** it into distinct, human-readable components. It transforms datetime data, autoregressive lags, and user-defined variables into mathematical basis functions and feeds them into standard machine learning regression models (such as Ridge regression). The primary components of the algorithm include:

Growth & Trend: Models how the series changes over the long term (e.g., linear or quadratic growth).

Seasonality: Breaks down recurring patterns (daily, weekly, monthly) using Fourier series.

Events & Holidays: Captures the specific impact of calendar events and holidays on the timeline.

Autoregression: Improves forecasts for shorter horizons (under 30 days) by using past values $Y(t-1)$, $Y(t-2)$ to predict future outcomes.

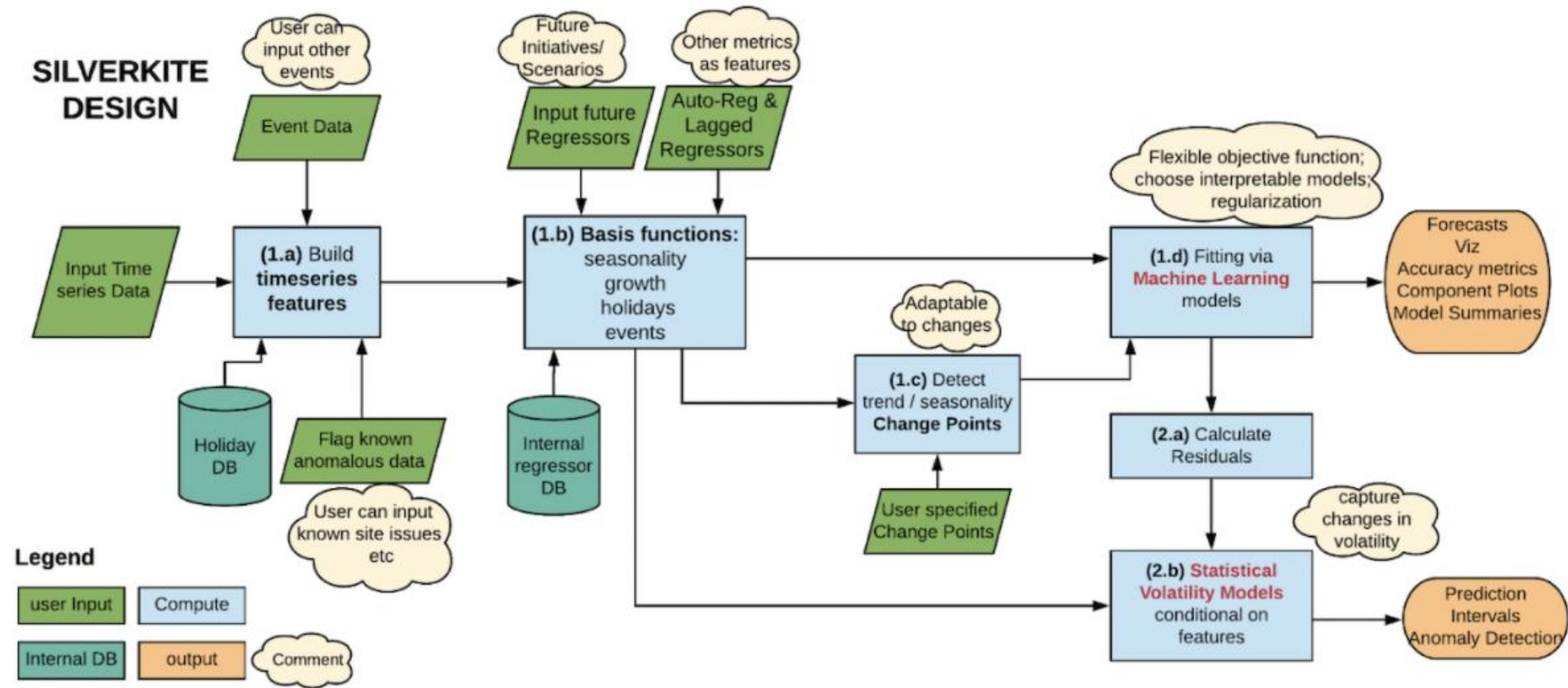


Figure 1. Architecture diagram for Greykite library's main forecasting algorithm, *Silverkite*

How Silverkite Works?

Raw time-series



Create **time**, **event**, **holiday**, **lag**, and **regressor** features



Represent the series using **trend**, **seasonality**, **holidays**, and **events**



Detect change **points**



Fit a **machine-learning** model



Generate forecasts and **accuracy reports**



Model residuals to **produce prediction intervals** and **anomaly detection**

So Silverkite is a **hybrid forecasting framework**: it combines **classical** time-series ideas like trend, seasonality, holidays, and change points with **machine-learning** models and uncertainty estimation.

Greykite Library

Greykite provides time-series **forecasting tools** for **trend, seasonality, holidays, and related effects**. It lets users fit preferred AI models and use templates for different forecasting needs, such as short-term hourly forecasts or long-term daily forecasts.

The library also **supports experimental plots**, tuning templates, and model customization. Users can mark known anomalies and decide whether to ignore, adjust, or keep them.

Greykite can **detect outliers** and **select the best model** from candidate configurations based on past performance. Instead of tuning each forecast manually, users can define several candidate setups that capture different trend patterns.

It also helps interpret the model. For example, it can **estimate holiday effects**, show the **impact of changepoints**, or explain how specific **features improve the forecast**.

Greykite is also designed to be **fast**. LinkedIn reports that a 7-day forecast trained on more than eight years of daily data can be produced in only a few seconds, with the full pipeline taking under 45 seconds.

Forecasting Workflow

- Collect time-series data
- Perform feature engineering
- Split data using walk-forward validation
- Train forecasting models
- Evaluate performance
- Deploy best-performing model

Key Takeaways

- Unsupervised learning enables anomaly and change detection.
- Detecting structural changes is crucial in real-world time-series systems
- Walk-forward validation is essential
- DTW improves similarity measurement
- Silverkite provides flexible forecasting
- XGBoost adapts regression to forecasting
- Ensemble models improve robustness
- Feature engineering remains critical